

PRD: ML-Enhanced FX Signal Engine (API + Backtesting)

1. Product Summary

A Python-based prototype that:

- Ingests real-time **FX price data** from an API.
- Builds a **lightweight ML model** to predict short-term direction.
- Generates **EA-style trade signals** (BUY/SELL/FLAT) with suggested SL/TP.
- Includes a **backtester** to evaluate strategy performance over historical data.

The goal is to mimic the **core brain of an Expert Advisor (EA)** without binding to a specific platform, showing:

- Machine learning integration
 - API/data integration
 - Backtesting & validation
 - EA-like configuration & outputs
-

2. Goals & Non-Goals

2.1 Goals (for v1)

1. **End-to-end pipeline** from data → features → model → signals → backtest.
2. **Single instrument** (e.g., EURUSD) on a fixed timeframe (e.g., 1h or 15m).
3. **Simple ML model**:
 - Logistic Regression / Random Forest (scikit-learn)
 - LSTM

- KAN

4. **EA-style signal interface:**

- JSON/CSV with fields like: `timestamp`, `signal`, `entry_price`, `stop_loss`, `take_profit`, `confidence`.

5. **Backtester:**

- Simulate trades based on signals.
- Output basic metrics: win rate, PnL, max drawdown, Sharpe-like ratio.

6. **CLI or script entrypoints to:**

- Fetch and store data.
- Train/update model.
- Run backtest and print a small report.

7. **Readable, commented code + README** showing how this aligns with:

- EA development,
- ML integration,
- Backtesting and optimization.

2.2 Non-Goals (v1)

- No real money execution, order routing, or broker integration.
- No full MQL/MetaTrader or TradingView integration (just EA-style behavior).
- No super-fancy frontend UI; CLI + simple matplotlib plots (if time) is enough.
- No hyperparam grid search beyond small, fixed configs.

3. Primary User & Use Cases

Primary “user”:

- Quant developer / ML engineer at a trading firm (your target persona for the job).

Key Use Cases

1. Train a new ML model on historical EURUSD data

- As a quant dev, I can run a script that:
 - pulls or loads historical OHLCV data,
 - engineers features,
 - trains a model,
 - saves the model to disk.

2. Generate EA-style trading signals for a historical data window

- I can load a saved model and produce signals (BUY/SELL/FLAT) for each bar.

3. Backtest a model+signal configuration

- I can run a backtest script that:
 - replays historical bars,
 - applies signals with simple execution assumptions,
 - outputs metrics and plots PnL over time.

4. Simulate real-time ingestion

- I can simulate a “live” loop over recent data:
 - fetch last N candles from API/local file,
 - generate current signal,
 - log it in a format compatible with an EA integration.
-

4. High-Level Architecture

Modules:

1. `data_api/`

- `data_fetcher.py`: fetches OHLCV data from an API (or loads CSV).
- `data_store.py`: handles caching/storing historical data locally (e.g., `data/eurusd_1h.csv`).

2. `features/`

- `feature_engineering.py`: transforms OHLCV into model features:
 - Returns a DataFrame with:
 - Price returns,
 - Rolling mean/volatility,
 - Simple indicators (SMA, RSI-lite, etc.),
 - Target label: $y = 1$ if next bar close > current close, else 0 .

3. `model/`

- `train_model.py`: trains model(s) and saves them (`models/model.joblib`).
- `predict.py`: loads model, returns predictions & probabilities.

4. `signals/`

- `signal_engine.py`: converts model predictions → EA-style signals:
 - `signal`: BUY/SELL/FLAT
 - `entry_price`: current close
 - `stop_loss`, `take_profit`: simple multiples of ATR or volatility.

- `confidence`: predicted probability.

5. `backtest/`

- `backtester.py`: takes:
 - historical data,
 - signals,
 - simple position sizing rules,
and returns performance metrics + trade list.

6. `cli/` or `scripts/`

- `run_train.py` – run E2E training.
- `run_backtest.py` – run E2E backtest and print metrics.
- Optional: `run_live_sim.py` – simulate streaming data.

5. Functional Requirements

5.1 Data Ingestion

- **FR-1**: Support loading historical OHLCV data from:
 - API response (e.g., JSON) or
 - CSV file in `data/` directory.
- **FR-2**: Normalize data into a standard schema:
 - Columns: `timestamp`, `open`, `high`, `low`, `close`, `volume`.
- **FR-3**: Allow configuration of:
 - Symbol (default: `EURUSD`),

- Timeframe (e.g., 1h, 15m) — even if v1 just supports one.

5.2 Feature Engineering

- **FR-4:** Compute a small, well-defined feature set, e.g.:
 - Lagged returns (1, 3, 5 bars),
 - Rolling mean & std of returns,
 - Simple moving averages (e.g., 10, 20),
 - Optional: cheap indicator like RSI or %K.
- **FR-5:** Generate a classification target:
 - $y = 1$ if `close[t+1] > close[t]`, else 0.
- **FR-6:** Handle NaNs by dropping initial rows or filling.

5.3 Model Training & Inference

- **FR-7:** Train at least one classifier:
 - Baseline: Logistic Regression (scikit-learn).
 - Optional: RandomForestClassifier.
- **FR-8:** Split data into train/validation (e.g., time-based split).
- **FR-9:** Save model + feature scaler to disk.

FR-10: Expose a prediction function:

```
def predict_signals(df_features: pd.DataFrame) -> pd.DataFrame
```

- returning probabilities and predicted class.

5.4 Signal Generation (EA-like)

- **FR-11:** Convert predictions to discrete signals:

- probability > `p_buy` → BUY
- probability < `p_sell` → SELL
- else → FLAT
(with configurable thresholds).

FR-12: For each bar, emit a structured signal:

```
{  
  "timestamp": "...",  
  "symbol": "EURUSD",  
  "signal": "BUY",  
  "entry_price": 1.0850,  
  "stop_loss": 1.0830,  
  "take_profit": 1.0890,  
  "confidence": 0.74  
}
```

-
- **FR-13:** Export signals to:

- CSV (`outputs/signals.csv`)
- JSON (`outputs/signals.json`).

5.5 Backtesting

- **FR-14:** Implement a simple vectorized/sequential backtester:

- Assumptions:
 - Enter at next bar's open,
 - Single position at a time, or
 - One trade per bar, flat at close.

- **FR-15:** Compute metrics:
 - Total PnL,
 - Win rate,
 - Max drawdown,
 - Number of trades,
 - Simple Sharpe proxy (mean/vol of returns).
- **FR-16:** Optional: Plot equity curve (using matplotlib).

5.6 Configuration & CLI

- **FR-17:** Define a config file (`config.yaml`) for:
 - symbol,
 - timeframe,
 - thresholds,
 - train/test split dates.
- **FR-18:** Provide simple CLI commands:
 - `python scripts/run_train.py --config config.yaml`
 - `python scripts/run_backtest.py --config config.yaml`

6. Non-Functional Requirements

- **NFR-1:** Run end-to-end on a laptop in < 1–2 minutes.
- **NFR-2:** Code should be clean, modular, and easy to extend.

- **NFR-3:** Minimal dependencies: `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `pyyaml`, maybe `requests`.
-

7. v1 vs Future Iterations

v1 (What you'll build now)

- Single symbol (EURUSD), single timeframe.
- Single ML model, simple configuration.
- Offline backtest using CSV or downloaded data.
- CLI + basic plots.

Future Ideas (to talk about in interviews)

- Multiple symbols, multiple timeframes.
 - LSTM/Temporal models (PyTorch or TensorFlow).
 - Real broker/MetaTrader integration.
 - Proper walk-forward or cross-validation.
 - Risk management module (position sizing, portfolio constraints).
-

8. Deliverables

1. **Codebase** with clear structure:
 - `data_api/`, `features/`, `model/`, `signals/`, `backtest/`, `scripts/`.
2. **Example notebooks or scripts** that:
 - train a model,

- generate signals,
- run backtest + print metrics.

3. **README.md:**

- How to run it,
- Example commands,
- Example metrics,
- Short section “How this maps to an Expert Advisor role.”